

WSS Final Report Team 5

Team Members and Contribution

Name	Contribution
Gabriel Azar	Idea generator
Marco Joson	Implemented WSS in Godot, final report formatting, contributed to final report
Jason Lin	Made Sprites, contributed Diagrams, contributed to report, made general report outline
Aidan Ma	Contributed Diagrams
Kian Naderi	Created/contributed a bunch of diagrams, including the main class diagram, added comments to code, contributed to final report
Gabren Sysavath	Made Sprites and contributed Diagrams

Brief Summary of What's Finished

In the project, we currently have implemented nearly all if not all of what was assigned to us. Terrain types were generated procedurally and implemented with varying costs depending on terrain you remained on. Multiple vision types to show what you could see were done. Traders and different types of traders/trading was implemented. Brain types that point to desired resources. It is also user controlled so no automatic gameplay. A map and sprites were made so it was easy to tell what tiles were what, what resources did what, etc. Additionally, it was also designed so you can play the game by choosing your own parameters such as map size, difficulty, brain, and vision which allows users creativity.

Object Oriented Design application

Object Oriented Design was applied and helped in our design, but not in the ways initially intended. The diagrams we made helped to give a guideline to how things should be implemented, and how things should relate to each other, however there were some slight differences in normal object oriented design versus the program used to make it: Godot. Since Godot is an engine to create games, a lot of the OOP principles had to be manually done (to show our understanding) or were displayed differently than traditional OOP.

Grading Information

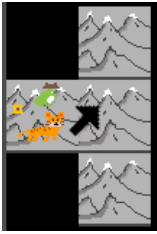
A. Types of terrain: 5

- Each terrain provides some sort of specific penalty
- Plains
 - strength -= 1
 - water -= 0
 - food -= 0
- Desert
 - strength -= 2
 - water -= 3
 - food -= 2
- Mountains
 - strength -= 3
 - water -= 1
 - food -= 3
- Swamp
 - strength -= 1
 - water -= 2
 - food -= 2
- Tundra
 - strength -= 3
 - water -= 1
 - food -= 1

B. Types of traders: 3

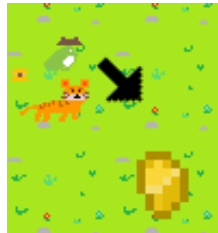
- Trader type and price is based on difficulty
- Behavior: You will propose a trade to the trader using the textboxes. You will input your gold and the desired food and water. The trader will evaluate the trade and if they accept the offer, you can press “accept” to finalize the trade. If they do not accept, the updates will update on your menu. You may exit any time.
- Dumb
 - Easy to accept deals and has the lowest gold to resource ratio
- Regular
 - Has the fairest deals and gold to resource ratio
- Angry
 - Has the most unfair gold to resource ratio

C. Types of vision: 4

Focused	Cautious	Keen-eyed	Far-sighted
			

D. Types of brains: 3

- Each brain will have an arrow that points to the nearest resource brain type desires AKA the type of brain.
- Greedy
 - Points to the nearest gold



- Thirsty
 - Points to the nearest water



- Hungry
 - Points to the nearest food

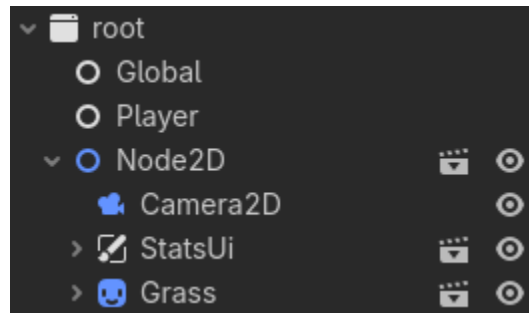


E. Video demonstration

- YouTube link: <https://youtu.be/Zud-BU3-j9Y>

F. Hierarchy example

- Hierarchy in Godot is shown in scene trees or in a parent-child relationship
- Example 1: Scene tree example found in Godot application



- Explanation: `root` is the root file running the game while `Global` and `Player` are global scripts and `Node2D` is the map node where every child under it is something that shows up on the map.
- Example 2: Parent-child relationship in code
 - the_map.gd line 265: `add_child(tile)`
 - Creates a tile child under the map node
 - the_map.gd line 274: `add_child(player_instance)`
 - Creates (spawns) the player under the map node

G. Inheritance example

- Inheritance is used when specialized classes extend a general base class
- Example 1: Trader
 - There is a base trader class: `Trader.gd`

```
extends Node2D

class_name BaseTrader

func evaluate_trade(gold_offered, requested_food, requested_water):
    >| return {
    >| >| "accepted": false,
    >| >| "counter_gold": gold_offered,
    >| >| "counter_food": requested_food,
    >| >| "counter_water": requested_water
    >| }
```

- Each trader type extends this base program
- Example with `regularTrader.gd`:

```

1  extends "res://Scripts/Trader.gd"
2
3  #will working until a fair trade
4  #1 gold = 2 food or 2 water or 1 of each
5  func evaluate_trade(gold_offered, requested_food, requested_water):
6      >|
7      >| var value_requested = requested_food + requested_water
8      >| var value_given = gold_offered * 2
9      >|
10     >| if value_given >= value_requested:
11     >| >| return {
12     >| >| >| "accepted": true,
13     >| >| >| "counter_gold": gold_offered,
14     >| >| >| "counter_food": requested_food,
15     >| >| >| "counter_water": requested_water}
16     >| >|
17     >| return {
18     >| >| "accepted": false,
19     >| >| "counter_gold": ceil(value_requested / 2.0),
20     >| >| "counter_food": requested_food,
21     >| >| "counter_water": requested_water}

```

- Example 2: Items
 - Base class: item.gd

```

1  extends Node2D
2  #This program is the main class for collecting items and updating player stats
3
4  #Called when player collects the item
5  func collect():
6      >| apply_effect()
7      >| queue_free() # remove item from scene
8
9  #To be overridden by child scripts
10 func apply_effect():
11     >| pass

```

- Inherited class example: goldItem.gd

```

1  extends "res://Scripts/item.gd"
2
3  func apply_effect():
4      >| Player.gold += 1
5
6  func get_item_type():
7      >| return "gold"

```

H. Polymorphism example

- Polymorphism takes place when different objects respond to same function differently
- Example 1: Line 102 - func find_nearest_desired_item(): in theBrain.gd

- It uses `child.get_item_type()` in line 119 which is different items (food, gold, water) has a different implementations of that function
- Example 2: In the same file in line 98: `func get_desired_item_type()`:

```
98  ▾ func get_desired_item_type():
99  >| return null
```

- This is initially null because the child brain scripts override this function completely such as found `thirstyBrain.gd`

```
1  extends "res://Scripts/theBrain.gd"
2
3  ▾ func get_desired_item_type():
4  >| return "water"
```

I. Encapsulation example

- Encapsulation is demonstrated by keeping data and related behavior together inside objects.
- Example 1: Movement cost
 - The player movement logic is encapsulated inside `player.gd` instead of being modified directly by outside scripts.
 - `player.gd` line 18 contains everything needed
 - `theBrain.gd` line 75

```
71  #get the movement cost of the movement per step
72  ▾ func apply_movement_cost(tile_pos: Vector2i):
73  >| var map = get_parent()
74  >| var terrain = map.map[tile_pos.x][tile_pos.y]
75  >| Player.apply_movement_cost(terrain)
```

- Example 2 Tile visibility
 - Tile visibility is done in `tileObject.gd` in line 13

```
12  #making tiles visible under vision
13  ▾ func set_visible_state(is_visible: bool):
14  ▾ >| if is_visible:
15  >| >| modulate = Color(1,1,1) # normal
16  >| >| $Button.disabled = false
17  ▾ >| else:
18  >| >| modulate = Color(0.001,0.001,0.001) # dark/clouded
19  >| >| $Button.disabled = true
```

- `the_map.gd` line 298 and 300

```
297  ▾ >| >| >| if player_instance.vision.is_tile_visible(player_tile, Vector2i(x,y))
298  >| >| >| >| tile.set_visible_state(true)
299  ▾ >| >| >| else:
300  >| >| >| >| tile.set_visible_state(false)
```

J. Abstraction example

- Abstraction hides implementation details behind simpler interfaces.
- Example 1: Using vision in theBrain.gd
 - Line 297

```
player_instance.vision.is_tile_visible(player_tile, Vector2i(x,y))
```

- theBrain.gd does not need to know how visibility calculations work. It simply calls the abstraction provided by the Vision class found in Vision.gd.

K. Good comments

```
# Placeholder function for child scripts to override
```

- We felt it was important to note that some of our functions are intended to be overridden by other scripts that may be using them in order for the code to make as much sense as it can.

```
# Focused vision shows the current tile and a small forward cone
```

```
# Cautious vision shows the current tile and nearby tiles in a cross shape
```

- Two examples of the comments used for the different vision types; we felt it was important to include these comments to describe what each vision type does and how it interacts with the game and the player

For fun:

```
# Dumb Trader is very easy to trade with (really is a dummy)
```

- This was a funny comment we wanted to include for the trader we decided to call Dumb Trader

L. Something impressive: Up to instructors interpretation

- You can interact / “play” for yourself
- User guide

NOTE: To all scripts are found in: wss/Scripts/[file.gd]